CS780 Capstone Project: OBELIX: the Warehouse Robot

Aritra Ambudh Dutta
230191
aritraad23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This report presents a comprehensive exploration of reinforcement learning algorithms for the OBELIX warehouse robot, tasked with finding, attaching to, and pushing a grey box to the arena boundary in a partially observable environment. Over four development phases, we systematically investigated seven distinct approaches spanning tabular Q-learning, Deep Q-Networks (DQN), Dueling Double Deep Recurrent Q-Networks (D3QN with GRU), Recurrent PPO, and hybrid heuristic-RL methods. Key innovations include potential-based reward shaping (PBRS), curriculum learning, frame stacking for temporal context, prioritized experience replay, and heuristic fallback policies for the zero-sensor exploration phase. Our best-performing agent achieved a score of -1447.55 on the final Codabench test phase using a Dueling D3QN with frame stacking and heuristic search.

1 Competition Result

Codabench Username: a_230191
Final leaderboard rank (Test Phase): [Fill in]
Leaderboard rank – Level 1 (static): [Fill in]
Leaderboard rank – Level 2 (blinking): [Fill in]
Leaderboard rank – Level 3 (moving + blinking): [Fill in]

Number of valid submissions made:

Phase	**Submissions**
Development – Level 1 (Static)	[Fill in]
Development – Level 2 (Blinking)	[Fill in]
Development – Level 3 (Moving + Blinking)	[Fill in]
Final Test Phase	80+

Table 1: Number of valid submissions per phase.

2 Introduction and Problem Description

The OBELIX project draws inspiration from the seminal 1991 work by Mahadevan and Connell [1], which demonstrated one of the earliest applications of reinforcement learning on a physical robot for box-pushing tasks. Our task is to design an RL-based autonomous controller for a simulated

mobile robot named OBELIX that must locate a grey box in an arena, attach to it, and push it to the arena boundary.

The environment presents a **Partially Observable Markov Decision Process (POMDP)** with the following characteristics:

- **Observations**: An 18-bit binary vector consisting of 8 near sonar sensors, 8 far sonar sensors, 1 infrared (IR) sensor for direct frontal detection, and 1 attachment/stuck indicator.
- **Actions**: 5 discrete actions – rotate left 45° (L45), rotate left 22.5° (L22), move forward (FW), rotate right 22.5° (R22), and rotate right 45° (R45).
- **Rewards**: Sparse and one-time sensor rewards (+1 for side sensors, +2 for far forward, +3 for near forward, +5 for IR), a large attachment bonus (+100), push penalty (−1/step while pushing), success bonus (+2000), wall collision penalty (−200), and idle penalty (−1/step with no sensor activation).

The task is evaluated across three difficulty levels of increasing complexity:

1. **Level 1 – Static Box**: The box remains stationary. The robot must navigate to it, attach, and push to boundary.
2. **Level 2 – Blinking Box**: The box randomly appears and disappears. The robot cannot attach when the box is invisible, but once attached, blinking stops.
3. **Level 3 – Moving + Blinking Box**: The box moves with constant velocity and also blinks. Movement stops once the robot attaches.

Additionally, a **vertical wall with a small opening** may appear in any level, introducing navigation obstacles with a harsh −200 penalty for wall collisions. The evaluation uses hidden random seeds across 10 runs per difficulty level, making overfitting to specific configurations infeasible.

The core challenges are: (1) **perceptual aliasing** – many world states map to identical observations (especially all-zeros when out of range); (2) **sparse rewards** – randomly finding, attaching, and pushing the box is astronomically unlikely; and (3) **exploration-exploitation trade-off** – systematic search when blind, efficient exploitation when sensors activate. Our goal was to maximize mean cumulative reward across all levels with wall robustness.

3 Approach and Model Evolution

Our work evolved significantly across the four development phases, with each iteration informed by the failures and insights of prior attempts.

3.1 Phase 1–2: Initial Exploration (Levels 1 and 2)

We explored six baseline algorithms: **(a) Standard DQN** (18→64→64→5 MLP, 1K–2K episodes) – consistently converged to a degenerate circling policy (L22↔R22), scoring ∼−1980; **(b) Tabular Q-Learning** (2^{18} = 262K states, up to 1M episodes) – slow convergence due to enormous state space; **(c) A2C** (shared actor-critic, 5K episodes) – marginally better exploration but still failed to find the box; **(d) Dueling DRQN** (GRU-based, 18→64→GRU(64)→5, 4-frame stack) – showed memory capability but still circled; **(e) DRQN with Anti-Rotation Shaping** (−0.2 penalty for 8+ consecutive turns, +1.5 FW bonus on zero sensors, 72-dim input through 16-step GRU) – best Phase 1–2 approach but still −1980 on Codabench; **(f) Dueling DDQN with Memory Augmentation** (22-dim input: 18 raw + visibility flag, steps-since-seen, sin/cos angle to box; 128-unit shared layers with dueling heads) – addressed blinking but not the core exploration problem.

3.1.1 Root Cause Analysis

After Phase 1–2, we identified four reasons for the persistent circling behavior: (1) **Perceptual aliasing** – many world states map to identical observations, especially the all-zeros case; (2) **Deceptive rewards** – one-time sensor rewards mean the dominant signal becomes -1/step, making spinning preferable to risky forward movement; (3) **Exploration failure** – ϵ-greedy cannot reliably find the box; (4) **Shaping distortion** – aggressive bonuses caused optimization for shaped rather than true rewards.

3.2 Phase 3: Structured Approaches (Level 3)

Based on our analysis, we developed four structured approaches:

Approach 1 – Recurrent PPO (RPPO) [2]: LSTM-based actor-critic (77-dim input: 4-frame stack + one-hot prev-action, 128-unit LSTM). Curriculum learning over 5 phases (spawn distance: 20px $\rightarrow$ random) with 7-component dense reward shaping. Performed well in training but poorly on Codabench's random spawns (-1500 to -4000).

Approach 2 – Dueling D3QN + GRU + PER [3, 4, 5]: 128-unit GRU with PER ($\alpha = 0.6$, β: $0.4 \rightarrow 1.0$ via SumTree) and 3-step returns. Same input/curriculum as Approach 1. Best single-episode score in Phase 3 (-1515) but inconsistent.

Approach 3 – Discrete SAC [6]: Entropy-regularized $J(\pi) = \mathbb{E}[Q(s,a) - \alpha \log \pi(a|s)]$ to prevent circling. Did not converge within Phase 3's limited training budget.

Approach 4 – Hybrid Heuristic + RL: FSM with hand-crafted rules for search/approach, NN fallback for edge cases. Pragmatic but insufficient for wall obstacles.

3.3 Phase 4: Final Approaches (Test Phase)

The critical insight from Phase 3 was that **curriculum learning distorted training** – agents trained near the box never learned to explore, and aggressive reward shaping caused optimization for shaped rewards rather than true environment rewards. This led to three refined approaches:

Approach 5 – Simple DDQN without Curriculum: A deliberate simplification. We used a 2-layer MLP ($77 \rightarrow 128 \rightarrow 128 \rightarrow 5$) Double DQN with **no curriculum learning** and **minimal reward shaping** (only anti-circling: -3.0 for 6+ consecutive turns, and anti-stuck: -10.0). The key innovation was a **heuristic exploration policy** at inference: when all sensors read zero, instead of relying on the neural network (which outputs near-random actions for the all-zeros state), the agent executes a deterministic "lawnmower" search pattern – 15 forward steps, then a 90° turn, alternating direction. Training used random spawns across all difficulties with wall obstacles always enabled, for 15,000 episodes.

Approach 6 – D3QN v2 with GRU, No Curriculum: Similar philosophy of minimal intervention applied to the recurrent architecture. Dueling D3QN with 128-unit GRU, 77-dim input, 16-step sequences. No curriculum, minimal shaping. The GRU hidden state was reset after 5 consecutive zero-observations to prevent saturation. Trained for 12,000 episodes across mixed difficulties.

Approach 7 – Refined D3QN with PBRS: Our most principled approach used Potential-Based Reward Shaping (PBRS) [7], which is theoretically guaranteed to preserve optimal policies.

Three shaping components were used:

$$\Phi_{\text{approach}}(s) = -\frac{d(\text{robot}, \text{box})}{d_{\max}} \tag{1}$$

$$\Phi_{\text{heading}}(s) = \cos(\theta_{\text{facing}} - \theta_{\text{to_box}}) \quad \text{(pre-attachment only)} \tag{2}$$

$$\Phi_{\text{push}}(s) = -\frac{d(\text{box}, \text{nearest_wall})}{d_{\max}} \quad \text{(post-attachment only)} \tag{3}$$

The shaped reward at each step was $F(s, s') = \gamma\Phi(s') - \Phi(s)$ with a scaling factor of 10.0. The architecture used a larger Dueling DQN (256→128 shared encoder) with 6-frame stacking (108-dim input), PER ($\alpha = 0.6$), and 3-step returns. Curriculum was reintroduced but more conservatively (4 phases over 8000 episodes). Trained with max_steps=2000.

3.4 Detailed Description of Best Models

3.4.1 Approach 1: Dueling D3QN + Heuristic (Best Score: -1447.55)

- **Algorithm**: Double DQN with frame stacking
- **Architecture**: MLP (77→128→128→5) with ReLU activations
- **State representation**: 4-frame stack (72 dims) + previous action one-hot (5 dims)
- **Exploration**: Heuristic lawnmower search when sensors=0; ϵ-greedy otherwise (ϵ: 1.0→0.05 over 500K steps)
- **Reward shaping**: Minimal – anti-circling (-3.0) and anti-stuck (-10.0) only
- **Inference**: Hybrid policy – heuristic for search, reactive rules for sensor-active states, NN only when confident (Q-value margin > 5.0)

3.4.2 Approach 7: D3QN + PBRS (Best Score: -1540.74)

- **Algorithm**: Dueling Double DQN with PER and n-step returns
- **Architecture**: Dueling MLP (108→256→128, then split into value head 128→64→1 and advantage head 128→64→5). $Q(s, a) = V(s) + A(s, a) - \text{mean}(A)$
- **State representation**: 6-frame stack (108 dims)
- **Exploration**: ϵ-greedy with slow decay over 200K steps
- **Reward shaping**: 3-component PBRS (approach, heading, push) with scale 10.0
- **Curriculum**: 4-phase progressive difficulty introduction

4 Experiments and Implementation Details

4.1 Training Setup

All experiments were conducted on a CPU-based environment (as required by the submission constraints) with Python 3 and PyTorch. Some training runs utilized GPU acceleration (CUDA) with mixed-precision training (AMP), but all inference code was designed to run on CPU with `map_location=''cpu''`.

4.2 Hyperparameter Summary

Parameter	DQN	DRQN	RPPO	D3QN	DDQN-v2	D3QN-v2	Refined
Learning rate	5e-4	2e-4	1e-4	1e-4	2e-4	1e-4	1e-4
γ	0.99	0.98	0.99	0.99	0.99	0.99	0.99
Batch size	64	64	–	64	64	64	64
Buffer size	100K	300K	–	75K	75K	75K	50K
ϵ decay	200K	200K	–	500K	500K	500K	200K
Target update	2K	1K	–	2K	2K	2K	2K
Episodes	1K	4K	8K	8K	15K	12K	8K
Max steps	1K	1K	1K	1K	2K	2K	2K
Input dim	18	72	77	77	77	77	108
Frame stack	1	4	4	4	4	4	6
Prev. action	No	No	Yes	Yes	Yes	Yes	No
Recurrence	No	GRU	LSTM	GRU	No	GRU	No
Curriculum	No	No	Yes	Yes	No	No	Yes
PER	No	No	No	Yes	No	No	Yes
n-step	1	1	1	3	1	1	3

Table 2: Hyperparameter comparison across all major approaches. "–" indicates not applicable (on-policy method).

4.3 Multi-Difficulty Training

In Phase 4 approaches, we trained on all three difficulty levels simultaneously with wall obstacles always enabled. Difficulty was sampled randomly each episode from a weighted distribution biased toward harder levels: $P(\text{Level } 0) = 2/7$, $P(\text{Level } 2) = 2/7$, $P(\text{Level } 3) = 3/7$. This ensured the agent did not overfit to a single difficulty.

4.4 Local Evaluation

To avoid overfitting to specific seeds, we evaluated each checkpoint on 10 runs per difficulty level with random seeds, matching the Codabench evaluation protocol. We used our custom `rank_checkpoints_codabench.py` script to automate this process and select the best checkpoint for submission.

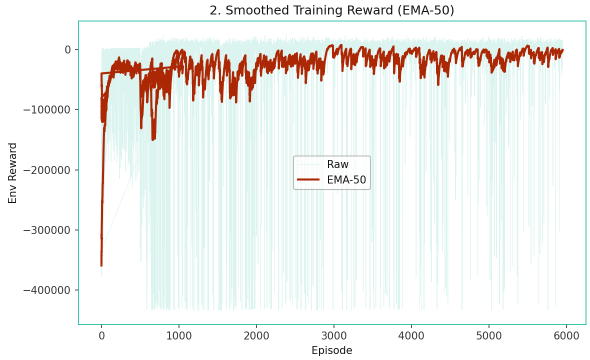
5 Results

5.1 Codabench Test Phase Results

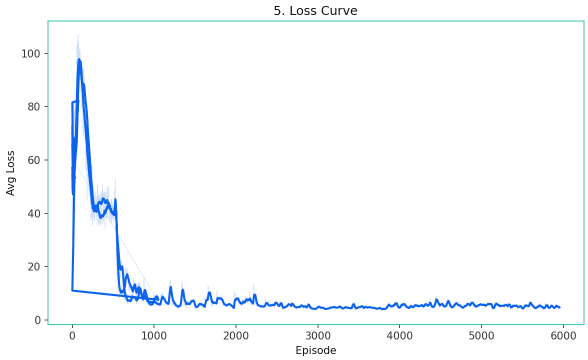
Approach	Best Checkpoint	Codabench Score
1 – Dueling D3QN + Heuristic	approach_1_ep7200	−1447.55
1 – Dueling D3QN + Heuristic	approach_1_ep7400	−1561.20
7 – Refined D3QN + PBRS	approach_7_ep05200	−1540.74
3 – D3QN + PER (Phase 3 arch)	approach_3_ep2000	−1655.64
1 – Dueling D3QN + Heuristic	approach_1_ep5200	−1738.57
3 – D3QN + PER	approach_3_ep6400–8000	−1827 to −1829
7 – Refined D3QN + PBRS	approach_7_ep02800	−1904.76
7 – Refined D3QN + PBRS	approach_7_ep04050	−2033.55
Dueling D3QN LSTM (Phase 3 baseline)	Dueling_D3QN__LSTM_V3	−1828.41
1 – Dueling D3QN + Heuristic	approach_1_ep4800	−4081.87
7 – Refined D3QN + PBRS	approach_7_ep03750	−6111.92

Table 3: Codabench test phase scores (sorted by score, higher is better). All scores are mean cumulative reward across 3 difficulty levels × 10 runs, with wall obstacles.

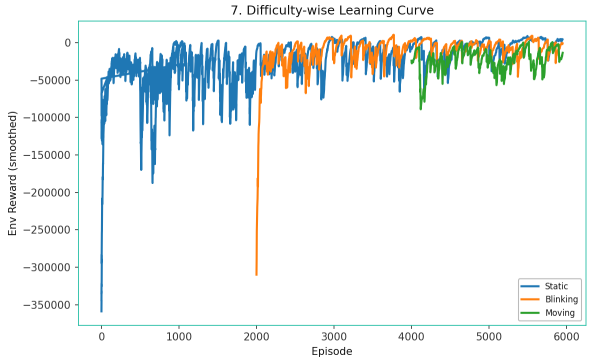
5.2 Training Curves (Approach 7 – Refined D3QN with PBRS)



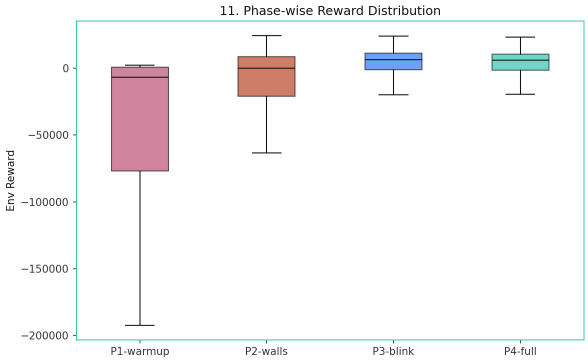
(a) Smoothed training reward (EMA-50). Reward improves sharply after curriculum Phase 1, then stabilizes with high variance.



(b) TD loss curve. Loss spikes during early episodes due to large reward magnitudes, then converges to a low stable value by episode 2000.



(c) Per-difficulty learning curves. Static (Level 0) learns fastest; blinking and moving levels introduced later via curriculum show slower improvement.



(d) Reward distribution per curriculum phase. Variance decreases and median reward improves across phases, indicating progressive learning.

Figure 1: Training diagnostics for Approach 7 (Refined D3QN with PBRS, 6000 episodes). All plots use true environment reward (not shaped reward).

5.3 Key Observations from Results

Approach 1 dominated the leaderboard with its best checkpoint (ep7200) achieving -1447.55. This is notable because it used the simplest architecture (feedforward MLP) – suggesting that for this POMDP, the heuristic search policy for zero-sensor states was more impactful than sophisticated neural architectures.

Approach 3 (D3QN with PER) showed remarkable consistency – nearly all checkpoints from ep2000 to ep8000 scored between -1827 and -1829, suggesting the agent converged to a stable policy. However, this policy was a local optimum (likely circling with occasional sensor-triggered actions).

Approach 7 (Refined D3QN with PBRS) showed the highest variance – scores ranged from -1540 (second best overall) to -6111 across different checkpoints. This indicates that PBRS created a complex loss landscape where checkpoint selection was critical.

Earlier checkpoints sometimes outperformed later ones, particularly for Approach 1

(ep7200 > ep15000). This suggests overfitting to training conditions at higher episode counts, possibly due to the exploration rate (ϵ) becoming too low.

5.4 Learning Dynamics

The training process revealed a characteristic pattern across all approaches:

1. **Episodes 0–2000**: High exploration ($\epsilon > 0.5$). Random policy occasionally finds the box, yielding sparse positive rewards. Training reward is highly noisy.
2. **Episodes 2000–6000**: ϵ decays, policy begins to stabilize. Two divergent outcomes: either the agent learns meaningful sensor-response behaviors, or it converges to the circling attractor.
3. **Episodes 6000+**: Low ϵ. Policy is largely fixed. Further training may degrade performance if the agent has overfit to the shaped reward landscape.

The critical transition occurred around episodes 5000–8000, where the best checkpoints typically emerged. Training beyond this point often decreased Codabench scores.

6 Error Analysis and Discussion

6.1 Common Failure Modes

Level 1 (Static Box): Primary failure was not *finding* the box when spawned far away. The lawnmower heuristic may not sweep the box within the 2000-step budget, and wall collisions (-200) compound the issue.

Level 2 (Blinking Box): The blinking introduces false negatives – the agent may be near the box but unable to detect it during invisible phases, leading to "orbiting" without attachment. One-time sensor rewards provide no signal for maintaining proximity after first detection.

Level 3 (Moving + Blinking): Creates a "pursuit" problem. Without a motion model, agents can only react to current sensors; if the box moves away, sensors deactivate and the agent reverts to search.

With Walls: The narrow opening requires precise alignment with coarse 22.5°/45° rotations. Multiple collisions can accumulate $-1000+$ in penalties per episode.

6.2 Why Simple Approaches Outperformed Complex Ones

Simpler architectures (MLP + heuristic) outperformed sophisticated ones (LSTM/GRU + PBRS) due to: (1) **recurrent state saturation** – GRU hidden states saturate after 8+ identical zero-observations; (2) **curriculum-evaluation mismatch** – curriculum-trained agents never learned to handle random spawns; (3) **reward shaping interference** – dense shaping dominated true environment signal; (4) **heuristic effectiveness** – a lawnmower pattern provides systematic arena coverage that random/learned exploration cannot match.

6.3 Limitations and Future Improvements

Key limitations and future directions include: **richer observations** (continuous sensor readings or learned latent states), **hierarchical RL** (high-level mode switching between search/approach/push), **MCTS** for planning in deterministic settings, and **intrinsic motivation** (RND or count-based exploration) to replace heuristic search.

7 Conclusion

This project provided deep insights into the practical challenges of applying RL to partially observable robotic tasks. Our key findings are:

1. **Simplicity can win**: In POMDPs with sparse rewards, a well-designed heuristic for the "unknown" state combined with a simple learned policy for "known" states can outperform end-to-end deep RL approaches.
2. **Reward shaping is a double-edged sword**: While theoretically principled (PBRS), reward shaping in practice can create deceptive gradients that lead to local optima. Minimal shaping (only penalizing clearly undesirable behaviors like circling) proved more robust.
3. **Curriculum learning requires careful design**: Starting easy can prevent the agent from ever learning to handle hard cases (random spawns, wall obstacles). Training without curriculum but with slower exploration decay was more robust for generalization.
4. **Checkpoint selection matters**: The best checkpoint was not always the last one. Systematic evaluation of intermediate checkpoints (every 200 episodes) was crucial for finding the best submission.

The key takeaway: in POMDPs with sparse rewards, the bottleneck is **exploration**, not the RL algorithm. Future work should focus on learned exploration, hierarchical decomposition, and richer state representations.

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?
  Claude (Anthropic) was used in a limited advisory capacity.

- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Clarifying PBRS theory, DRQN architecture de
Ideation / brainstorming	Yes	Generating list of possible approaches (rl_brains
Debugging small code snippets	No	
Plotting / visualization code	No	
Grammar / writing style	Yes	Proofreading report for clarity
Report structure / section ideas	Yes	Organizing content into required sections
Other (specify)	No	

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
  Yes: (1) Sutton & Barto textbook [8], (2) Original OBELIX paper by Mahadevan & Connell [1], (3) Double DQN paper [4], (4) Dueling DQN paper [3], (5) PPO paper [2], (6) PER paper [5], (7) PBRS paper by Ng et al. [7], (8) DRQN paper by Hausknecht & Stone [9].

- Apart from any LLM usage, what was your own contribution?
  All algorithmic implementation, training, hyperparameter tuning, debugging, and experimental evaluation were performed independently. The core contributions include: (1) implementing and training 7 distinct RL approaches from scratch using PyTorch based on lecture pseudocode and paper descriptions; (2) designing the heuristic fallback search policy (lawnmower pattern) based on manual play observations; (3) conducting systematic root cause analysis of the circling failure mode; (4) developing the multi-difficulty training strategy;

(5) building automated checkpoint ranking and submission pipelines; (6) all experimental analysis and conclusions drawn from the results.

References

[1] S. Mahadevan and J. Connell, "Automatic programming of behavior-based robots using reinforcement learning," *Artificial Intelligence*, vol. 55, no. 2-3, pp. 311–365, 1992.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," *Proceedings of the 33rd International Conference on Machine Learning*, pp. 1995–2003, 2016.

[4] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[5] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

[6] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *Proceedings of the 35th International Conference on Machine Learning*, pp. 1861–1870, 2018.

[7] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," *Proceedings of the 16th International Conference on Machine Learning*, pp. 278–287, 1999.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction.* MIT Press, 2nd ed., 2018.

[9] M. Hausknecht and P. Stone, "Deep recurrent q-learning for partially observable MDPs," *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.